AK45-36 KV80 + ROS 2 Control

Setup Guide — Waveshare USB-CAN-A on ROS 2 Humble

---

1. How It All Works

The Physical Layer

The AK45-36 communicates over a CAN bus — a two-wire protocol (CAN-H and CAN-L) designed for reliable communication in noisy environments like robots and vehicles. Every device on the bus has a unique ID; in this setup the motor is ID 111. The Waveshare USB-CAN-A adapter bridges the CAN bus to your PC over USB.

The Waveshare Adapter Problem

Linux has a standard system for CAN hardware called SocketCAN, which makes a CAN interface appear as a network device (e.g. can0) that software can open like a socket. Most CAN adapters support this natively — but the Waveshare USB-CAN-A does not. Instead it uses a proprietary serial protocol where every CAN frame is wrapped in 0xAA...0x55 bytes and sent over a virtual COM port (/dev/ttyUSB0).

The cubemars_hardware ROS 2 package only knows how to talk SocketCAN. To bridge the gap, a Python script (waveshare_can_bridge.py) was written that reads the Waveshare serial frames, unpacks them, and reinjects them into a virtual SocketCAN interface (vcan0) created in software. It also does the reverse for outgoing commands.

Full Stack Diagram

```
Motor (AK45-36)
    |  CAN bus (physical wires, CAN-H and CAN-L)
    v
Waveshare USB-CAN-A
    |  USB -> /dev/ttyUSB0  (proprietary 0xAA...0x55 serial frames)
    v
waveshare_can_bridge.py
    |  translates to/from standard SocketCAN frames
    v
vcan0  (virtual SocketCAN interface in the Linux kernel)
    |  standard SocketCAN socket
    v
cubemars_hardware  (ROS 2 hardware interface plugin)
    |  reads position / velocity / effort, sends position commands
    v
ros2_control  (controller manager, runs at 100 Hz)
    |
    v
joint_trajectory_controller
    |  accepts trajectory goals
    v
Your ROS 2 code
```

Bugs Fixed in cubemars_hardware

1. Kernel CAN filter missing CAN_EFF_FLAG. The AK45-36 uses extended (29-bit) CAN frames. The Linux kernel requires CAN_EFF_FLAG to be set when filtering for extended frames — without it, all frames were silently dropped before the C++ code ever saw them. The filter was removed entirely and ID

matching is handled in software instead.

2. MSG_DONTWAIT causing immediate EAGAIN. The recv() call used MSG_DONTWAIT, which returns immediately if no frame is available at that exact microsecond. Since the controller runs at 100 Hz and the motor broadcasts at ~50 Hz, the timing rarely lined up. Removing MSG_DONTWAIT and adding a 10 ms socket timeout means the call now waits briefly for a frame to arrive, which it always does.

3. Status frame ID prefix not stripped. The motor sends status frames with ID 0x296F — the 0x29 is a status-type prefix and 0x6F is motor ID 111. The package was trying to match the full raw ID against just 111, so it never found a match. Masking with & 0xFF when reading strips the upper byte, leaving just the motor ID.

---

2. Starting the Full Stack

Everything is brought up with a single script. Make sure the motor is powered and the Waveshare adapter is plugged in before running it.

```
~/mars_ws/start.sh
```

> This script: creates the vcan0 virtual interface, starts the bridge, sources ROS 2, and launches the controller manager with all controllers active. On Ctrl+C it cleans up the bridge process and removes vcan0.

---

3. Sending Position Commands

Command Structure

Commands are sent by publishing a JointTrajectory message to the controller topic. Positions are in **radians**. time_from_start is how long the motor has to reach that position.

```
ros2 topic pub /joint_trajectory_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory "{
    joint_names: [ak45_joint],
    points: [{positions: [1.57], time_from_start: {sec: 2}}]
}" --once
```

Common Position Values

```
0.0      =   0 degrees  (home / zero)
0.7854   =  45 degrees
1.5708   =  90 degrees
3.1416   = 180 degrees
-1.5708  = -90 degrees
```

Multi-Waypoint Trajectory

You can chain multiple waypoints — the motor moves smoothly through each one:

```
ros2 topic pub /joint_trajectory_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory "{
    joint_names: [ak45_joint],
    points: [
      {positions: [0.0],    time_from_start: {sec: 1}},
      {positions: [1.5708], time_from_start: {sec: 3}},
      {positions: [0.0],    time_from_start: {sec: 5}}
    ]
}" --once
```

Monitoring Joint State

```
ros2 topic echo /joint_states
```

This streams live position (rad), velocity (rad/s) and effort (Nm) from the motor.

---

4. Using Three Daisy-Chained Motors

CAN is a bus protocol — all three motors connect to the same CAN-H and CAN-L wires. Each motor needs a unique ID set via the R-Link (e.g. 111, 112, 113). The bridge and vcan0 handle all three transparently with no changes needed.

> **Important:** the CAN bus requires a 120 ohm termination resistor at each physical end of the cable. The Waveshare adapter terminates one end. The last motor in the chain should have its onboard termination enabled — check the AK45-36 driver board for a termination jumper or solder bridge.

URDF ros2_control Section (three joints)

```
<ros2_control name="ak45_system" type="system">
  <hardware>
    <plugin>cubemars_hardware/CubeMarsSystemHardware</plugin>
    <param name="can_interface">vcan0</param>
  </hardware>

  <joint name="joint_1">
    <param name="can_id">111</param>
    <param name="pole_pairs">14</param>
    <param name="gear_ratio">36</param>
    <param name="kt">0.11</param>
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>

  <joint name="joint_2">
    <param name="can_id">112</param>
    <param name="pole_pairs">14</param>
    <param name="gear_ratio">36</param>
    <param name="kt">0.11</param>
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>

  <joint name="joint_3">
    <param name="can_id">113</param>
    <param name="pole_pairs">14</param>
    <param name="gear_ratio">36</param>
    <param name="kt">0.11</param>
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>
</ros2_control>
```

controllers.yaml (three joints)

```
controller_manager:
  ros__parameters:
    update_rate: 100
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster
    joint_trajectory_controller:
      type: joint_trajectory_controller/JointTrajectoryController

joint_trajectory_controller:
  ros__parameters:
    joints:
      - joint_1
      - joint_2
      - joint_3
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
```

Commanding all three motors simultaneously

```
ros2 topic pub /joint_trajectory_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory "{
    joint_names: [joint_1, joint_2, joint_3],
    points: [{
      positions: [1.57, 0.78, -1.57],
      time_from_start: {sec: 2}
    }]
}" --once
```

This moves joint_1 to 90 deg, joint_2 to 45 deg, and joint_3 to -90 deg simultaneously.

---

5. AK45-36 KV80 Key Parameters

```
Pole pairs:      14
Gear ratio:      36:1
Kt (Nm/A):       0.11
Peak torque:     24 Nm
Rated torque:    8 Nm
No-load speed:   52 rpm (output shaft)
Rated voltage:   24 V
CAN bitrate:     1 Mbit/s
CAN ID:          Set via R-Link (default 1, set to 111 for this setup)
Status frame ID: 0x29 << 8 | motor_id  (e.g. 0x296F for ID 111)
```